

# Pandemic modelling with the game of life

Esame Metodi Computazionali 19/06/2026

---

Brancaccio Lorenzo

Celentano Francesco Maria

Il generalized semi-classical game of life (gSCGOL) è un automa cellulare descritto tramite qubit i quali vengono evoluti tramite ripetute applicazioni di combinazioni lineari degli operatori di nascita  $\hat{B}$ , morte  $\hat{D}$  e sopravvivenza  $\hat{S}$ . Si è usato il modello per descrivere l'interazione umano-virus durante la pandemia da COVID-19.

$$|\psi\rangle = a|1\rangle + b|0\rangle = \begin{pmatrix} a \\ b \end{pmatrix}$$

$$\hat{B} = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}, \quad \hat{D} = \begin{pmatrix} 0 & 0 \\ 1 & 1 \end{pmatrix}, \quad \hat{S} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

- Griglia  $401 \times 401$   
 $\times 2$

- $\hat{G} |\psi\rangle = \begin{pmatrix} a' \\ b' \end{pmatrix}$

- $a'^2 + b'^2 = 1$

| Regole di $\hat{G}$         |                                                                  |
|-----------------------------|------------------------------------------------------------------|
| $A$                         | $\hat{G}$                                                        |
| $A \leq A_0 - 2V$           | $\hat{D}$                                                        |
| $A_0 - 2V < A \leq A_0 - V$ | $(\sqrt{2} + 1)[(A_0 - V) - A]\hat{D} + [A - (A_0 - 2V)]\hat{S}$ |
| $A_0 - V < A \leq A_0$      | $(\sqrt{2} + 1)[A_0 - A]\hat{S} + [A - (A_0 - V)]\hat{B}$        |
| $A_0 < A \leq A_0 + V$      | $(\sqrt{2} + 1)[(A_0 + V) - A]\hat{B} + [A - A_0]\hat{D}$        |
| $A \geq A_0 + V$            | $\hat{D}$                                                        |

$$A_i = \sum_{k=1}^8 a_{ki}$$

# Regole gSCGOL: Virus

- Griglia  $401 \times 401$  di stati classici
- Aggiungere 100 virus il giorno  $D_v$  in posizioni randomiche sulla griglia
- $N_i = v_p X_i a_i$  virus aggiunti ogni giorno (dove  $v_p$  e  $a_i$  sono parametri)
- Un virus aggiunto il giorno  $n$  nella cella  $i$  non può infettare fino a  $n + v_e + 1$ . Resta infettivo fino a  $n + v_e + v_l$  e viene rimosso al giorno  $n + v_e + v_l + 1$
- Un virus non può vivere al di fuori di un corpo per più di 3 giorni consecutivi
- Ogni  $V_s$  giorni viene aggiunto un virus casuale nella griglia
- Ogni cella non può contenere più di  $V_m$  virus

## Parametri del modello

| Parameter | Value |
|-----------|-------|
| $s$       | 0.341 |
| $v_p$     | 0.23  |
| $v_e$     | 3     |
| $v_l$     | 13    |
| $D_v$     | 52    |
| $v_m$     | 950   |
| $v_s$     | 5     |

$$X_i = \sum_{k=1}^8 v_{ki}$$

# Andamento dei decessi teorico e sperimentale

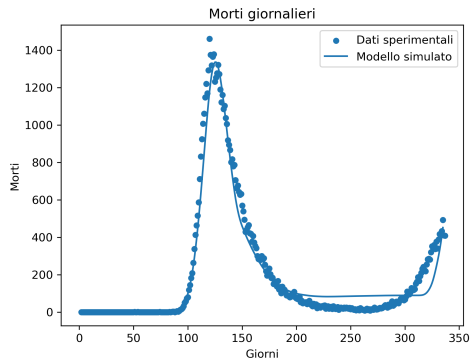
Ogni infezione del virus causa delle morti nei giorni successivi secondo:

$$P(x) = \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}$$

dove  $\alpha = 4.938$  e  $\beta = 0.2627$ . Dopo  $g$  giorni il numero di morti è:

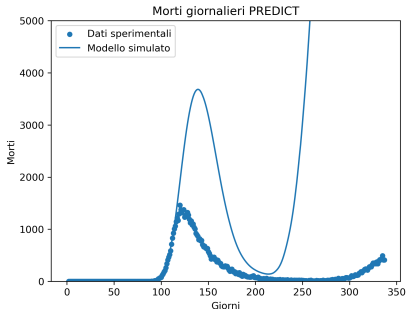
$$D(g) = S \sum_{\tau=1}^{31} I(g - \tau) P(\tau)$$

(3 giorni prima del test positivo e 28 giorni per la letalità del virus)

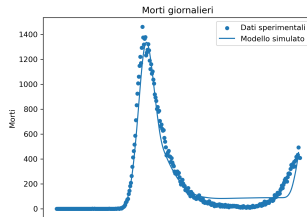


# Trial and Error e forma di V

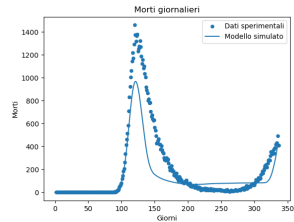
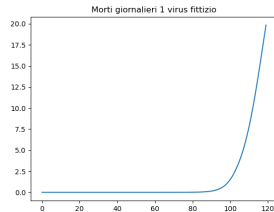
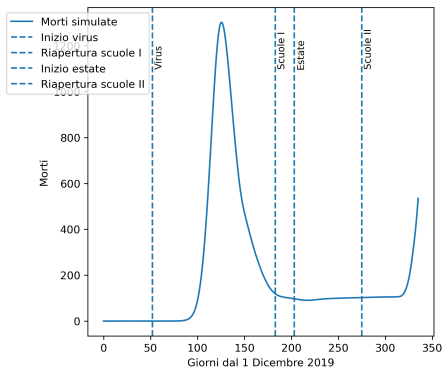
```
def V(giorno):  
    if giorno <= 113: return 1  
    elif giorno <= 207: return 0.85  
    elif giorno <= 218: return 0.009 * giorno - 0.185  
    elif giorno <= 248: return 0.002 * giorno + 0.688  
    elif giorno <= 278: return 0.007 * giorno - 0.092  
    else: return 1.2
```



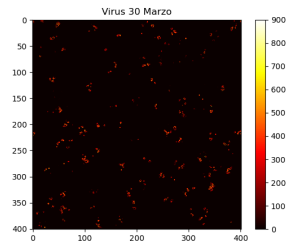
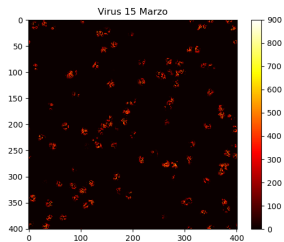
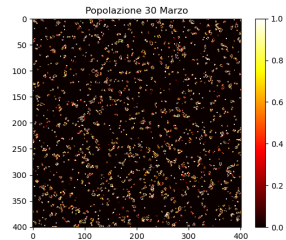
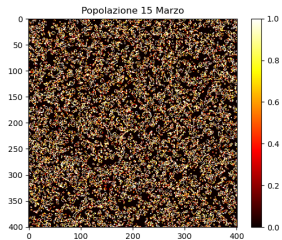
```
def V(giorno):  
    if giorno <= 90: return 1  
    elif giorno <= 106: return 0.9  
    elif giorno <= 113: return 0.85  
    elif giorno <= 120: return 0.75  
    elif giorno <= 134: return 0.88  
    elif giorno <= 141: return 0.86  
    elif giorno <= 155: return 0.85  
    elif giorno <= 180: return 0.86  
    elif giorno <= 190: return 0.85  
    elif giorno <= 211: return 0.8  
    elif giorno <= 230: return 0.75  
    elif giorno <= 252: return 0.8  
    elif giorno <= 259: return 0.8  
    elif giorno <= 267: return 0.9  
    elif giorno <= 308: return 1.0  
    elif giorno <= 312: return 1.1  
    else: return 1.2
```



# Predizioni e risultati



# Conclusioni





# Applicazione del modello all'Italia

Parametri cambiati:

$$V_p = 0.23 \longrightarrow V_p = 0.25$$

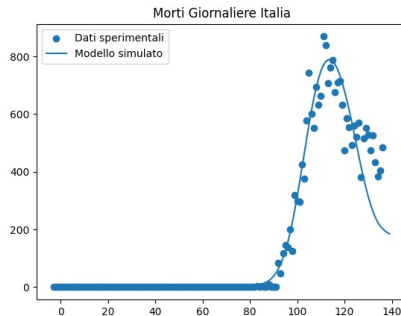
$$Dv = 52 \longrightarrow Dv = 48$$

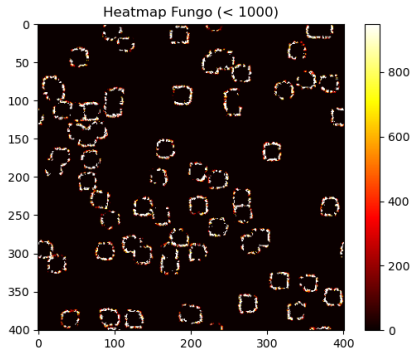
$$\alpha = 4.938 \longrightarrow \alpha = 4.0$$

$$\beta = 0.2627 \longrightarrow \beta = 0.19$$

Tasso di mortalità in UK=140-150 decessi per 100.000 abitanti

Tasso di mortalità in Italia=110-130 per 100.000 abitanti





Con memoria

```
VIRUS_LAYERS = np.zeros((giorni_totali, 401, 401))  
VIRUS_LAYERS[Dv].flat[indici] = 1  
VIRUS_LAYERS[i] += N  
VIRUS_INFETTI = (VIRUS_LAYERS[:,i] * infettivi_mask).sum(axis=0)  
VIRUS_LAYERS[:,i+1] = np.where(eta_full > Ve + VL, 0, VIRUS_LAYERS[:,i+1])
```

Senza memoria

```
VIRUS = np.zeros((401, 401))  
GIORNO_MASK = np.full((401, 401), -999)  
N = Vp * X_i * STORICO[i]  
VIRUS = VIRUS + N  
giorni_vita = i - GIORNO_MASK  
VIRUS = np.where(giorni_vita > Ve + VL, 0, VIRUS)
```